BASH SCRIPTING CHEATSHEET

DEVOPS QUICK REFERENCE • PRACTICAL • PORTABLE • PRODUCTION READY

1 SHEBANG & EXECUTION

1.1 Shebang (Interpreter Selection)

Type	Syntax	Notes
Portable	`#!/usr/bin/env bash`	Works across systems ✅
Fixed	`#!/bin/bash`	Depends on system path ⚠️

MEMORY: `env` → finds interpreter via PATH
`bin` → fixed path

1.2 Execution Methods

Method	Command	Behavior
Make executable	`chmod +x script.sh`	Adds execute permission
Run (kernel)	`./script.sh`	Uses shebang + kernel
Run (direct)	`bash script.sh`	Ignores shebang, uses bash directly

1.3 Requirements Comparison

Command	chmod required	shebang required
`./script.sh`	✅ Yes	✅ Yes
`bash script.sh`	❌ No	❌ No

1.4 Why ./script.sh Needs Both

Requirement	Reason
chmod	Kernel checks execute permission
shebang	Kernel needs interpreter to run script

DevOps Tip: Always use `#!/usr/bin/env bash` in CI/CD scripts to avoid path issues across environments.

2 EXECUTION FLOW (INTERNAL)

User runs script
↓
Shell
↓
Kernel (execve)
↓
Checks executable bit (chmod)
↓
Reads shebang (#!)
↓
Invokes interpreter (bash)
↓
Script executes

Key Difference

`./script.sh` → kernel + shebang involved

`bash script.sh` → ignores shebang, uses bash directly

DevOps Tip: If a script fails mysteriously, run it with `bash -x script.sh` to trace execution step-by-step.

3 VARIABLES & QUOTING

Task	Syntax	Example
Declare	`VAR="val"`	`NAME="DevOps"`
Use (safe)	`"$VAR"`	`echo "$NAME"`
No expansion	`'$VAR'`	prints $VAR
Export (env var)	`export VAR=value`	makes variable available to child processes

Best Practice

`"$VAR"` or `"${VAR}"`
→ prevents word splitting & globbing

Example

❌ Bad (unquoted)

```
FILE="*.txt"
rm $FILE   # expands unexpectedly
```

✅ Good (quoted)

```
FILE="*.txt"
rm "$FILE"   # safe
```

DevOps Tip: Unquoted variables are a common cause of production bugs—always quote variables in scripts.

4 INPUT HANDLING

Type	Syntax	Behavior
Basic input	`read VAR`	Reads user input
Prompt input	`read -p "Enter: " VAR`	Shows message
Safe input	`read -r VAR`	Preserves \

Important: -r Flag

❌ Without -r	`read VAR` `# hello\world → helloworld`
✅ With -r	`read -r VAR` `# hello\world → hello\world`

DevOps Tip: Use `read -r` when handling file paths or user input to prevent escape character issues.

5 COMMENTS

Type	Syntax / Example
Single line	`# this is a comment`
Inline	`echo "Hello"  # say hello`

Multi-line Comment (Hack)

```
: '\
  This is ignored
  Multiple lines
  '
```

Note: Fails if content has single quotes.

Block Ignore (Here-doc Trick)

```
: <<EOF
  This is ignored
  Multiple lines
  EOF
```

Rule: Start and end tag must match EXACTLY

DevOps Tip: Document why something is done—not just what—especially in automation scripts.

6 ENV & INPUT BEHAVIOR

Feature	Example	Output / Behavior
Env variable	`export NAME="DevOps"`	DevOps
User input	`read NAME` (user types Malathi)	Malathi
Prompt input	`read -p "Name: " NAME`	user enters
Safe read	`read -r VAR`	keeps \
Multi-comment	`: 'text'`	ignored
Block ignore	`: <<EOF ... EOF`	ignored

DevOps Tip: Use `export` only when needed—exposing unnecessary variables can cause side effects in child processes.

7 PORTABILITY RULE

Scenario	Use
CI/CD / GitHub / DevOps	`#!/usr/bin/env bash`
Fixed system (controlled)	`#!/bin/bash`

DevOps Tip: Test scripts in minimal environments (like containers) to catch portability issues early.

8 EXIT STATUS

`$?` # last command status

Code	Meaning
0	Success
≠ 0	Failure

Examples

```
command
if [ $? -eq 0 ]; then
  echo "Success"
else
  echo "Failed"
fi
```

DevOps Tip: Always check exit codes in automation scripts to avoid silent failures.

9 STRICT MODE

`set -euo pipefail`

Flag	Meaning
-e	exit immediately if a command exits with non-zero status
-u	treat unset variables as an error
-o pipefail	pipeline returns failure if any command in the pipeline fails

DevOps Tip: Use strict mode in all production scripts to prevent hidden failures.

🔥 FINAL MEMORY BLOCK (INTERVIEW READY)

Shebang
- `#!/usr/bin/env bash` → portable
- `#!/bin/bash` → fixed

Execution
- `./script.sh` → needs chmod + shebang
- `bash script.sh` → ignores shebang, uses bash directly

Flow
User → Shell → Kernel (execve) → Checks permission → Shebang → Interpreter → Script executes

Variables
- `"$VAR"` or `"${VAR}"` prevents word splitting & globbing
- `export VAR=value` → inherited by child processes (subshells)

Input
- `read` → input
- `read -p` → prompt
- `read -r` → safe (\ preserved)

Comments
- `#` → single line
- `: '...'` → multi-line (simple hack)
- `: <<EOF ... EOF` → block ignore

EXTRA QUICK REFERENCE

$#	Number of arguments
$@	All arguments
$1	First argument
$$	Current process ID
pwd	Print working directory
ls -l	List files
echo	Print text

DevOps Tip: Before production, always run `shellcheck script.sh` to catch hidden bugs and bad practices.

BASH SCRIPTING CHEATSHEET

ARGUMENTS • EXIT STATUS • STRICT MODE

1 WHAT IS "INPUT"?

Input = arguments passed to a script

```
./script.sh file.txt hello 123
```

Variable	Value
$0	script name (./script.sh)
$1	file.txt
$2	hello
$3	123

Rule: Everything AFTER script name = inputs

2 ARGUMENT VARIABLES

Purpose	Syntax	Example
Script name	$0	./script.sh
First argument	$1	file.txt
Second argument	$2	hello
Third argument	$3	123
Total arguments	$#	3
All args (safe)	"$@"	preserves all
All args (risky)	$*	word-split
All args (quoted)	"$*"	single string

3 ALL ARGUMENTS (CRITICAL)

Syntax	Behavior
$*	all args → subject to word splitting & globbing
"$*"	single string (joined by IFS, usually space)
"$@"	best → preserves each argument

Golden Rule: Always use "$@"

4 EXAMPLE: WHY "$@" MATTERS

Command

```
./script.sh "file one.txt" hello
```

SAFE: USING "$@"

```
for arg in "$@"; do
  echo "$arg"
done
```

Output

```
file one.txt
hello
```

Each argument stays intact

RISKY: USING $* (unquoted)

```
for arg in $*; do
  echo "$arg"
done
```

Output

```
file
one.txt
hello
```

Word splitting happens (broken)

MINI WORKING EXAMPLE

```
#!/usr/bin/env bash
echo "Script: $0"
echo "Count: $#"

for arg in "$@"; do
 echo "Arg: $arg"
done
```

Run

```
./script.sh "file one.txt" hello
```

Output

```
Script: ./script.sh
Count: 2
Arg: file one.txt
Arg: hello
```

MEMORY BLOCK (ARGUMENTS)

$0 → script name | $1 → first input | $# → number of inputs | "$@" → ALWAYS USE (safe) | $* → AVOID (splits args)

5 EXIT STATUS / EXIT CODE

WHAT IS $?

$? = exit status of the last command

```
echo $?
```

- Stores exit status of the last executed command

STANDARD MEANING

Code	Meaning
0	Success ✓
1–255	Failure ✗

IMPORTANT

0 = success
non-zero = error

USING exit

Stops script + returns status

Command	Meaning
exit 0	Stop script with success (Terminates with 0)
exit 1	Stop script with failure (Terminates with 1)
exit	Exits with last command's status

EXAMPLES

SUCCESS

```
ls
echo $?
```

Output: 0

FAILURE

```
ls wrongfile
echo $?
```

Output: 2 (non-zero)

REAL SCRIPT EXAMPLE

```
if [ ! -f "$1" ]; then
 echo "File not found"
 exit 1
fi

echo "File exists"
exit 0
```

KEY DIFFERENCE

$? → check last result (What just happened?)

exit N → set result & stop script (What I want to return?)

MEMORY BLOCK (EXIT STATUS)

$? → what just happened?
exit 0 → success & stop
exit 1 → failure & stop

0 = GOOD
1+ = BAD

6 STRICT MODE (set -euo pipefail)

WHAT IS IT?

Enables safe Bash execution (fail-fast mode)

```
set -euo pipefail
```

BREAKDOWN

Option	Meaning
-e	exit immediately if any command fails
-u	error on using undefined variables
-o pipefail	fail if any command in a pipeline fails

EXAMPLE

```
#!/usr/bin/env bash
set -euo pipefail

ls
ls wrongfile   # script stops here

echo "This will NOT run"
```

PIPELINE BEHAVIOR

```
cat file.txt | grep "text"
```

Without pipefail → may succeed even if cat fails ✗

With pipefail → fails correctly ✓

WHY IT MATTERS

- Prevents silent failures
- Stops broken scripts early
- Avoids hidden bugs in production

MEMORY

-e → stop on error
-u → no undefined variables
pipefail → pipeline must fully succeed

NOTE

```
grep "text" file || echo "not found"
```

This is safe (script won't exit)

DEVOPS RULE

```
#!/usr/bin/env bash
set -euo pipefail
```

Always place at the top of production scripts

QUICK RECAP

$0	Script name	"$@"	Use this always (safe)	$?	Check last result
$1	First input	$*	Avoid (splits args)	exit 0	Success
$#	Count			exit 1	Failure

DEVOPS TIP

Clean scripts + proper checks = fewer bugs, safe automation, happy production! 🚀

BASH TEST CONDITIONS CHEATSHEET

CHECK CONDITIONS • MAKE DECISIONS • WRITE SMART SCRIPTS

1 WHAT IS TEST CONDITION?

Used to evaluate true / false

`[ condition ]` `[[ condition ]]`

Used in:

```
if [ condition ]; then
    # code
fi
```

Result:
0 = true (success)
1 = false (failure)

2 BASIC TESTS

Check	Syntax	Example	Meaning
Empty	`[ -z "$a" ]`	`$a=""`	true if empty
Not empty	`[ -n "$a" ]`	`$a="hi"`	has value
File	`[ -f file ]`	file.txt	regular file exists
Directory	`[ -d dir ]`	mydir	directory exists
Exists	`[ -e file ]`	any	file/dir exists

3 FILE PERMISSIONS

Check	Syntax	Meaning
Readable	`[ -r file ]`	can read
Writable	`[ -w file ]`	can write
Executable	`[ -x file ]`	can execute
Not empty file	`[ -s file ]`	size > 0

Tip: These check the file's permissions and properties.

4 STRING COMPARISON

Check	Syntax	Example
Equal	`[ "$a" = "$b" ]`	strings same
Not equal	`[ "$a" != "$b" ]`	strings differ

Always quote strings:

```
[ "$a" = "$b" ]    # safe
```

Use [[ ]] when possible (safer than [ ])

5 INTEGER COMPARISON

Check	Syntax	Meaning
Equal	`[ "$a" -eq "$b" ]`	==
Not equal	`[ "$a" -ne "$b" ]`	!=
Less	`[ "$a" -lt "$b" ]`	<
Greater	`[ "$a" -gt "$b" ]`	>
Less or equal	`[ "$a" -le "$b" ]`	≤
Greater or equal	`[ "$a" -ge "$b" ]`	≥

Quotes are safe even for numbers (prevents empty variable issues).

6 [[ ]] (BASH SAFE TEST)

Feature	Syntax	Example	Meaning
Compare	`[[ "$a" == "$b" ]]`	$a=$b	safer (no word splitting)
Empty	`[[ -z "$a" ]]`	same as [ ]	cleaner syntax
Regex	`[[ "$a" =~ ^dev ]]`	^dev	pattern match

Do NOT quote regex pattern!

```
[[ "$a" =~ ^dev ]]      # ✅ correct
[[ "$a" =~ "^dev" ]]    # ❌ wrong
```

7 [ ] VS [[ ]] (IMPORTANT)

Type	Syntax	Use / Notes
POSIX (portable)	`[ "$a" = "$b" ]`	Works everywhere (sh, bash, etc.)
Bash (safer)	`[[ "$a" == "$b" ]]`	Preferred in bash (no word splitting, supports regex)
Regex support	`[[ "$a" =~ ^dev ]]`	Only in [[ ]] (advanced matching)

Use [[ ]] in bash for safety and advanced features.

8 ALL TESTS (QUICK REFERENCE)

STRING / VARIABLE

```
[ -z "$a" ]     # empty
[ -n "$a" ]     # not empty
```

FILE / DIRECTORY

```
[ -f file ]     # file
[ -d file ]     # directory
[ -e file ]     # exists
```

STRING COMPARISON

```
[ "$a" = "$b" ]     # equal
[ "$a" != "$b" ]    # not equal
```

INTEGER COMPARISON

```
[ "$a" -eq "$b" ]   # equal
[ "$a" -ne "$b" ]   # not equal
[ "$a" -lt "$b" ]   # less
[ "$a" -gt "$b" ]   # greater
[ "$a" -le "$b" ]   # less or equal
[ "$a" -ge "$b" ]   # greater or equal
```

BASH (SAFE) TEST

```
[[ "$a" == "$b" ]]   # compare
[[ -z "$a" ]]        # empty
[[ -n "$a" ]]        # not empty
[[ "$a" =~ regex ]]  # regex match
```

Memory:
[[ ]] = safer
(no word splitting, supports regex)

9 PRACTICAL EXAMPLES

1 STRING CHECK

```
a="dev"
b="dev"

if [ "$a" = "$b" ]; then
  echo "Equal"
fi
```

Compares two strings

2 INTEGER CHECK

```
a=5
b=10

if [ "$a" -lt "$b" ]; then
  echo "a is smaller"
fi
```

Compares numbers

3 FILE CHECK

```
if [ -f "file.txt" ]; then
  echo "File exists"
fi
```

Checks if file exists

4 DIRECTORY CHECK

```
if [ -d "mydir" ]; then
  echo "Directory exists"
fi
```

Checks if directory exists

5 REGEX (BASH ONLY)

```
a="devops"

if [[ "$a" =~ ^dev ]]; then
  echo "Starts with dev"
fi
```

Pattern match (regex)

Do NOT quote regex!

FINAL MEMORY BLOCK

`[ ]` → POSIX (portable) | `[[ ]]` → Bash (safer, regex)

-z → empty	-f → file	= → string equal	-eq → ==	-le → ≤
-n → not empty	-d → dir	!= → string not equal	-lt → <	-ge → ≥
	-e → exists		-gt → >	

Always quote strings: "$a" | Use [[ ]] when possible (safer, no word splitting, supports regex)

BEST PRACTICES

- ☑ Always quote variables: "$a"
- ☑ Prefer [[ ]] in bash scripts
- ☑ Do not quote regex pattern
- ☑ Use -z for empty check, -n for not empty
- ☑ Test files/directories before using them

DEVOPS TIP

Good test conditions prevent bugs and make scripts reliable in production.

```
$ ./script.sh
All checks passed! ✅
```

BASH CONTROL FLOW CHEATSHEET

1 LOGICAL OPERATORS

Topic	Syntax	Example	Meaning
AND	cmd1 && cmd2	mkdir dir && cd dir	run cmd2 if cmd1 succeeds
OR	cmd1 || cmd2	mkdir dir || echo "fail"	run cmd2 if cmd1 exits non-zero
NOT	! cmd	! grep -q "a" file	negates exit status

How it works

- cmd1 exits with 0 → run cmd2 (&&)
- cmd1 exits non-zero → run cmd2 (||)
- ! cmd → flips exit status

Note:
grep -q → silent (only exit status matters)

2 IF CONDITIONS

Topic	Syntax	Meaning
If	if [cond]; then	check condition
Else if	elif [cond]; then	additional check
Else	else	fallback
End	fi	close block

Example

```
if [ -f "file.txt" ]; then
  echo "File exists"
elif [ -d "dir" ]; then
  echo "Directory exists"
else
  echo "Not found"
fi
```

Note:
[ ] → for general conditions (files, strings)
(( )) → for numeric comparisons (preferred)

3 CASE (SWITCH STYLE)

Topic	Syntax	Meaning
Case start	case "$var" in	switch variable
Option	pattern)	matches value
Default	*)	fallback
End	esac	close

Example

```
case "$1" in
  start) echo "Starting";;
  stop)  echo "Stopping";;
  *)     echo "Invalid";;
esac
```

4 LOOPS

Types

Topic	Syntax	Meaning
For (list)	for i in a b; do	iterate values
For (range)	for i in {1..5}; do	sequence
C-style	for ((i=0;i<3;i++))	numeric loop
While	while [cond]; do	run while condition true
Until	until [cond]; do	run until condition true

Examples

range

```
for i in {1..5}; do
  echo "$i"
done
```

C-style

```
for ((i=0;i<3;i++)); do
  echo "$i"
done
```

while (safe)

```
while (( i < 3 )); do
  echo "$i"
  ((i++))
done
```

until (safe)

```
until (( i > 3 )); do
  echo "$i"
  ((i++))
done
```

5 LOOP CONTROL

Topic	Syntax	Meaning
Break	break	stop loop
Continue	continue	skip current iteration

Example

```
for i in {1..5}; do
  if [ "$i" -eq 3 ]; then
    continue
  fi
  if [ "$i" -eq 5 ]; then
    break
  fi
  echo "$i"
done
```

6 SAFE LOOP PATTERNS (IMPORTANT)

Topic	Syntax	Meaning
Files loop	for f in *.log; do	iterate files safely*
Read file	while read -r line; do	safe line-by-line read
Process input	while read -r line; do	process stream

Examples

safe file loop

```
shopt -s nullglob
for f in *.log; do
  echo "$f"
done
shopt -u nullglob
```

Note:
nullglob → avoids literal *.log if no files match

safe file read

```
while read -r line; do
  echo "$line"
done < file.txt
```

Note:
-r → prevents backslash escaping

better than pipe + ls

```
for file in *; do
  echo "$file"
done
```

Note:
Pipes run in subshell → variables won't persist outside loop

Avoid (unsafe)

```
for f in $(ls); do
  echo "$f"
done
```

Problem:
Breaks on spaces, tabs, newlines → unsafe for filenames
Caused by word splitting & globbing

7 BEST PRACTICES

- Always quote variables: "$var" → prevents word splitting & bugs
- Avoid $(ls)
- Prefer (( )) for numeric conditions
- Use -r with read

FINAL MEMORY BLOCK

LOGIC
&& → run if exit = 0
|| → run if non-zero
! → negate

IF
if → elif → else → fi

CASE
case → pattern) → *) → esac

LOOPS:
- for → list / range
- for (( )) → numeric loop
- while → run while condition is true
- until → run until condition becomes true

CONTROL
- break → stop
- continue → skip

SAFE
- for f in *.log → use with nullglob
- while read -r → safe input
- avoid $(ls)
- quote variables "$var"

BASH ADVANCED CHEATSHEET (FUNCTIONS → EXPANSION)

FUNCTIONS

Definition & Usage

Topic	Syntax	Example	Meaning
Define	func() {}	say() { echo "$1"; }	create function
Call	func	say hello	execute function
Argument	$1	first input	passed value
Return	return	return 0	exit status (0–255, mod 256)
Output	echo	echo data	actual output
Local var	local x=1	inside function	limited scope

Example

```
say() {
  local name="$1"
  echo "Hello $name"
  return 0
}

say "DevOps"
```

Output:

```
Hello DevOps
```

Important Difference

- return → exit status (0–255, mod 256)
- echo → actual data/output

2 FUNCTION VALIDATION

```
[ $# -ne 2 ] && return 1
```

Meaning:

- If number of arguments ≠ 2 → fail
- Non-zero return = failure

Example

```
add() {
  [ $# -ne 2 ] && return 1
  echo $(($1 + $2))
}

add 2 3   # 5
add 2     # fails (returns non-zero)
```

COMMAND SUBSTITUTION

Topic	Syntax	Example	Meaning
Capture output	VAR=$(cmd)	DATE=$(date)	store command result

Example

```
now=$(date)
echo "$now"
```

Output:

```
current date/time
```

Notes:

- Use $(cmd) instead of `cmd` (modern & nestable)
- $(cmd) strips trailing newline

ARITHMETIC

Topic	Syntax	Example	Meaning
Calculation	$(( ))	echo $((2+3))	math
Increment	((i++))	i=1 → 2	+1
Add assign	((i+=2))	i=1 → 3	add
Old style	let	let a=5+3	legacy

Example

```
a=5
b=3
echo $((a+b))    # 8

i=1
((i++))
echo $i          # 2
```

Note: ((expr)) → exit status 0 if result ≠ 0, else 1

ARRAYS

Topic	Syntax	Example	Meaning
Declare	arr=(a b c)	array	list
First element	${arr[0]}	a	index
All elements	${arr[@]}	a b c	full list
Length	${#arr[@]}	3	size

Examples

```
arr=(a b c)

echo "${arr[0]}"     # a
echo "${arr[@]}"     # a b c
echo "${#arr[@]}"    # 3

for i in "${arr[@]}"; do
  echo "$i"
done
```

{ } PARAMETER EXPANSION

Topic	Syntax	Example	Meaning
Default	${VAR:-x}	fallback	if unset OR null
Required	${VAR:?err}	error	must exist
Length	${#VAR}	size	count
Uppercase	${VAR^^}	caps	convert
Lowercase	${VAR,,}	small	convert
Substring	${VAR:0:3}	slice	part
Replace	${VAR/a/b}	replace	first match
Replace all	${VAR//a/b}	replace all	all matches

Examples

```
VAR="devops"

echo "${VAR:-default}"  # devops
echo "${#VAR}"          # 6
echo "${VAR^^}"         # DEVOPS
echo "${VAR,,}"         # devops
echo "${VAR:0:3}"       # dev
echo "${VAR/o/X}"       # devXps
```

Required Variable

```
echo "${VAR:?VARn is required}"
```

If VAR is unset or null → prints error & exits (non-interactive shell)

7 IFS (INPUT FIELD SPLIT)

Topic	Syntax	Example	Meaning
Split input	IFS=',' read	CSV	split values

Example

```
IFS=',' read -r a b c <<< "x,y,z"

echo "$a"    # x
echo "$b"    # y
echo "$c"    # z
```

Note: IFS here applies only to this command (temporary)

8 REDIRECTION & PIPES

Topic	Syntax	Example	Meaning
Overwrite	>	echo hi > f	write
Append	>>	echo hi >> f	add
Error	2>	cmd 2> err.txt	stderr only
All output	&>	cmd &> out.txt	stdout + stderr (bash-specific)
Pipe	|	ls | grep ".sh"	pipe output

Example

```
echo "hi" > file.txt
echo "again" >> file.txt

ls wrong 2> err.txt
ls &> all.txt

ls | grep ".sh"
```

Note:
&> is bash-specific. For POSIX sh, use > out.txt 2>&1

FINAL MEMORY BLOCK

FUNCTIONS:

- func() {} → define
- $1 → input
- echo → output
- return → exit status (0–255, mod 256)

VALIDATION:

[$# -ne 2] → check args (non-zero = fail)

COMMAND:

VAR=$(cmd) → capture output

MATH:

- $(( )) → calc
- ((i++)) → increment

ARRAY:

- arr=(a b)
- ${arr[0]} → first
- ${arr[@]} → all
- ${#arr[@]} → size

{ } EXPANSION:

- ${VAR:-x} → default (unset or null)
- ${VAR:?err} → required
- ${#VAR} → length
- ${VAR^^} → upper
- ${VAR:0:3} → slice
- ${VAR//a/b} → replace all

IFS:

- IFS=',' → split (temporary)

REDIRECTION:

- > overwrite
- >> append
- 2> error
- &> both (bash-specific)
- | pipe

BASH TEXT PROCESSING & IO CHEATSHEET

1. TEXT PROCESSING (grep / awk / sed)

GREP (SEARCH)

Command	Example	Meaning
Case insensitive	`grep -i text file`	ignore case
Recursive	`grep -r text .`	search recursively (includes subdirectories)
Count	`grep -c text file`	count matching lines
Line number	`grep -n text file`	show line numbers
Invert match	`grep -v text file`	exclude matches
Regex OR	`grep -E "a|b"`	regex OR (a or b)
Whole word	`grep -w text file`	match exact word
Only match	`grep -o text file`	print matched part only

Examples

```
grep -i "error" log.txt
# find "error" (ignore case)

grep -r "main" .
# search in current directory

grep -n "fail" file.txt
# show line numbers
```

AWK (COLUMN / PATTERN)

Command	Example	Meaning
Print column	`awk '{print $1}' file`	first column
Full line	`awk '{print $0}' file`	entire line
Custom delimiter	`awk -F: '{print $1}' file`	split by :
Pattern match	`awk '/pattern/ {print}' file`	filter lines

Example

```
awk '{print $1}' file.txt
awk -F: '{print $1}'
/etc/passwd  # usernames
```

SED (EDIT / REPLACE)

Command	Example	Meaning
Replace	`sed 's/a/b/g' file`	replace all matches per line (not across lines)
In-place	`sed -i 's/a/b/g' file`	modify file directly
Delete line	`sed '2d' file`	delete line 2

Example

```
sed 's/error/success/g'
file.txt
sed -i '2d' file.txt
```

2. FILE UTILITIES

Tool	Syntax	Example	Meaning
Cut	`cut -d: -f1 file`	`cut -d: -f1 file.txt`	first field extract column
Range	`cut -d: -f1-3 file`	`cut -d: -f1-3 file.txt`	fields 1–3 range extract
Sort numeric	`sort -n file`	`sort -n numbers.txt`	numbers numeric ascending
Sort reverse	`sort -r file`	`sort -r file.txt`	reverse descending
Sort by key	`sort -k2 file`	`sort -k2 file.txt`	column 2 sort by column
1,2,3 Count unique	`sort file | uniq -c`	`sort file.txt | uniq -c`	grouped count duplicates
Unique only	`sort -u file`	`sort -u file.txt`	remove duplicates unique lines
aA Translate	`tr 'a-z' 'A-Z"`	`tr 'a-z' 'A-Z'`	upper case change (stdin)
X Delete chars	`tr -d '0-9'`	`tr -d '0-9'`	remove digits filter characters
# Line count	`wc -l file`	`wc -l file.txt`	lines count lines
W Word count	`wc -w file`	`wc -w file.txt`	words count words
Head	`head -n 5 file`	`head -n 5 file.txt`	first lines preview start
Tail	`tail -n 5 file`	`tail -n 5 file.txt`	last lines preview end
Live tail	`tail -f file`	`tail -f log.txt`	live logs follow updates

Notes

- uniq works on consecutive duplicates → usually requires sorted input
- tr works on stdin (use with pipe or < file)

Examples

```
cut -d: -f1 file.txt
sort -n numbers.txt
sort file.txt | uniq -c
tr 'a-z' 'A-Z' < file.txt
wc -l file.txt
head -n 5 file.txt
tail -f log.txt
```

3. REDIRECTION & FILE DESCRIPTORS (FD)

Basic Redirection

Syntax	Example	Meaning
Overwrite	`> file`	`cmd > file.txt`
Append	`>> file`	`cmd >> file.txt`
Error only	`2> err.log`	`ls wrong 2> error.log`
All output	`&> file`	`ls &> all.log`
Merge	`> out 2>&1`	`cmd > out.txt 2>&1` stdout (order matters)

Example

```
echo "hi" > file.txt
echo "more" >> file.txt
ls wrong 2> error.log
ls &> all.log
cmd > out.txt 2>&1
```

Advanced File Descriptors

```
exec 3> file
echo "hi" >&3
exec 3>&-
```

Meaning:

- 3 → custom file descriptor
- >&3 → write to FD 3
- 3>&- → close FD 3

4. PIPES & STREAMS

Topic	Syntax	Meaning
Pipe	`cmd1 | cmd2`	pass output from cmd1 to cmd2
Tee	`cmd | tee file`	save output to file and print
Tee (append)	`cmd | tee -a file`	append output to file and print
Xargs	`cmd | xargs command`	pass input as arguments to command

Examples

```
ls | grep ".sh"
echo "hi" | tee file.txt
echo "hi" | tee -a file.txt
cat files.txt | xargs -r rm
find . -name "*.txt" -print0 |
 xargs -0 rm
```

Notes

- xargs runs command with input as arguments
- xargs -r → avoid running on empty input
- xargs -0 → safe for spaces/newlines

5. HERE DOCUMENT (MULTI-LINE INPUT)

Topic	Syntax	Meaning
Multi-line	`<<EOF ... EOF`	inline multi-line input

Examples

```
cat <<EOF
Hello $USER
EOF
```

👉 $USER will expand

```
cat <<'EOF'
Hello $USER
EOF
```

👉 $USER will NOT expand

Rules

- Start and end tag must match exactly
- End marker (EOF) must start at beginning of line (no spaces)
- <<'EOF' disables variable expansion

🔥 FINAL MEMORY BLOCK

GREP:
- -i ignore case
- -r recursive
- -n line no
- -v exclude
- -E regex OR
- -w whole word
- -o only match

AWK:
- $1 → column
- $0 → full line
- -F → delimiter

SED:
- s/a/b/g → replace (per line)
- -i → edit file

UTILS:
- cut → column
- -f1-3 → range
- sort → order
- sort -k → column sort
- uniq → unique/count (needs sorted input)
- tr → transform (stdin)
- wc → count
- head/tail → view

REDIRECTION:
- > overwrite
- >> append
- 2> error
- &> all (bash-only)
- 2>&1 merge (order matters)

PIPE:
- | → pass output
- tee → save + print
- tee -a → append
- xargs → arguments
- xargs -0 → safe input

FD:
- exec 3> → custom FD
- >&3 → write
- 3>&- → close

HERE DOC:
- <<EOF ... EOF → multi-line input
- <<'EOF' ... EOF → no variable expansion

BASH PROCESS, JOBS, SYSTEM & CRON CHEATSHEET

1. PROCESS MANAGEMENT

Task	Command	Example	Meaning
List processes	`ps aux`	`ps aux`	Snapshot of all running processes (point-in-time)
Detailed list	`ps -ef`	`ps -ef | grep nginx`	Filter/search processes
Tree view	`pstree`	`pstree`	Parent-child hierarchy
Live monitor	`top`	`top`	Real-time system stats (CPU, memory, etc.)
Better monitor	`htop`	`htop`	Interactive monitor (if installed)
Find PID by name	`pgrep`	`pgrep nginx`	Returns matching PID(s) (can be multiple)
Kill by PID	`kill`	`kill 1234`	Graceful stop (SIGTERM by default)
Force kill	`kill -9`	`kill -9 1234`	Force stop (SIGKILL, cannot be caught)
Kill by name	`pkill`	`pkill nginx`	Kill processes by name
Kill all (by name)	`killall`	`killall nginx`	Kill all matching processes (behavior may vary across Unix systems)

Signals

Signal	Usage	Meaning
SIGTERM (15)	`kill PID`	Graceful termination (default)
SIGKILL (9)	`kill -9 PID`	Immediate force kill (cannot be caught)
SIGINT (2)	`Ctrl + C`	Interrupt from terminal
SIGHUP (1)	`kill -1 PID`	Reload / terminal disconnect

2. JOB CONTROL (Shell Level)

Task	Command	Example	Meaning
Run in background	`cmd &`	`sleep 100 &`	Async execution
List jobs	`jobs`	`jobs`	Show jobs in current shell session only
Bring to foreground	`fg`	`fg %1`	Resume job in foreground
Resume in background	`bg`	`bg %1`	Continue stopped job in background
Suspend job	`Ctrl + Z`	—	Pause current foreground job
Wait for jobs	`wait`	`wait`	Wait for all (or specified) background jobs

3. SUBSHELL vs CURRENT SHELL

Type	Syntax	Meaning
Subshell	`(cmd)`	Runs in isolated environment (no variable persistence)
Current shell chaining	`cmd1 && cmd2`	Runs in same shell (executes cmd2 only if cmd1 succeeds)

4. PROCESS SUBSTITUTION

```
diff <(ls dir1) <(ls dir2)
```

👉 Treats command output as temporary file-like inputs

5. SYSTEM / DEVOPS COMMANDS

Task	Command	Meaning
Disk usage	`df -h`	Filesystem disk usage
Directory size	`du -sh *`	Size of each file/folder in current directory
Memory	`free -h`	RAM usage
CPU/process	`top / ps`	System activity
Uptime	`uptime`	System load & running time
Hostname	`hostname`	System name
Network IP	`ip a`	Network interfaces and assigned IPs
Open ports	`ss -tuln`	Listening ports

Systemd (Services)

Task	Command	Meaning
Status	`systemctl status nginx`	Detailed service status
Check active	`systemctl is-active nginx`	Returns service state (active/inactive)
Start	`systemctl start nginx`	Start service
Stop	`systemctl stop nginx`	Stop service
Restart	`systemctl restart nginx`	Restart service
Enable at boot	`systemctl enable nginx`	Enable auto-start on boot

6. CRON (AUTOMATION)

Task	Command
Edit cron	`crontab -e`
List cron	`crontab -l`
Remove cron	`crontab -r`

Time Format

```
* * * * *
```

- Day of week (0–7) (0 or 7 = Sunday)
- Month (1–12)
- Day of month (1–31)
- Hour (0 – 23)
- Minute (0—59)

Examples

```
0 2 * * *    /path/script.sh    # Daily at 2 AM
*/5 * * * *  /path/script.sh    # Every 5 minutes
0 0 * * 0    /path/script.sh    # Weekly (Sunday midnight)
```

Best Practice

- `/path/script.sh >> /var/log/script.log 2>&1`
- `>>` → append output
- `2>&1` → redirect stderr to stdout
- Always use full paths in cron
- Cron runs with minimal environment (no PATH, aliases, etc.)

7. ENV & COMMAND CONTROL

Task	Command	Meaning
Export variable	`export VAR=value`	Set environment variable
Bypass alias	`command ls`	Run actual command binary

8. ERROR HANDLING (SAFE SCRIPTING)

Option	Command	Meaning
Exit on error	`set -e`	Exit on error (with some exceptions)
Undefined variable	`set -u`	Error on unset variables
Pipe failure	`set -o pipefail`	Detect failures in pipelines
Debug mode	`set -x`	Trace command execution

Strict Mode (Best Practice)

```
set -euo pipefail
```

Fallback Handling

```
cmd || echo "fail"
```

9. TRAPS & SIGNAL HANDLING

Task	Command	Meaning
On exit	`trap 'cmd' EXIT`	Run on script exit
On Ctrl+C	`trap 'cmd' SIGINT`	Handle interrupt
Multiple signals	`trap 'cmd' SIGINT SIGTERM`	Handle multiple signals

Example

```
trap 'echo "Cleaning up..."' EXIT
```

10. FILE DESCRIPTORS (ADVANCED)

```
exec 3> file
echo "log" >&3
exec 3>&-
```

👉 Create, use, and close a custom file descriptor

What it means?

- `3>` → open file as FD 3 for writing
- `>&3` → write to FD 3
- `3>&-` → close FD 3

FINAL MEMORY BLOCK

PROCESS

- `ps aux` → list
- `pgrep` → find PID(s)
- `kill` → graceful stop
- `kill -9` → force kill
- `pkill/killall` → by name

JOBS

- `&` → background
- `jobs` → current shell jobs
- `fg/bg` → control
- `wait` → sync jobs

SYSTEM

- `df -h` → disk
- `free -h` → memory
- `top` → system monitor
- `ip a` → network
- `ss -tuln` → ports

CRON

- `crontab -e` → edit
- `*/5 * * * *` → every 5 min

ERROR

- `set -euo pipefail` → strict mode
- `cmd ||` → fallback

TRAP

- `trap EXIT` → cleanup
- `trap SIGINT` → Ctrl+C

ADVANCED

- `(cmd)` → subshell
- `<(cmd)` → process substitution
- `exec 3>` → file descriptor

BASH DEVOPS PATTERNS & AUTOMATION

1. SOURCE vs EXECUTION

Topic	Syntax	Meaning
Source	`source script.sh`	Runs in same shell (shares variables & environment)
Exec	`./script.sh`	Runs in subshell (child process) (isolated changes)
Exec	`bash script.sh`	Also runs in subshell (same as `./script.sh`)

Example:

```
VAR=hello
source script.sh      # VAR persists
./script.sh           # VAR not available after
bash script.sh        # same as above (subshell)
```

Memory:

- `source` → same shell (share env)
- `./script` / `bash` → subshell (child process)

2. ROOT & DEPENDENCY CHECK

Task	Syntax	Meaning
Root check	`[ "$EUID" -ne 0 ] && { echo "Run as root"; exit 1; }`	Require root/sudo
Command exists	`command -v git >/dev/null 2>&1`	Check dependency

Example:

```
command -v git >/dev/null 2>&1 || { echo "git not installed"; exit 1; }
```

3. PACKAGE MANAGER DETECTION

```
if command -v apt-get >/dev/null 2>&1; then
   echo "Debian/Ubuntu"
elif command -v dnf >/dev/null 2>&1; then
   echo "Fedora/RHEL (new)"
elif command -v yum >/dev/null 2>&1; then
   echo "RHEL/CentOS (old)"
elif command -v pacman >/dev/null 2>&1; then
   echo "Arch Linux"
else
   echo "Unknown package manager"
fi
```

4. IDEMPOTENCY (SAFE OPERATIONS)

Pattern	Meaning
`check || install`	Only install if not already present

Example (Debian-based):

```
dpkg -s nginx >/dev/null 2>&1 || apt-get install -y nginx
```

Portable pattern:

```
command -v nginx >/dev/null 2>&1 || install_nginx
```

Meaning:

- Already installed → skip
- Not installed → install

5. LOGGING (BEST PRACTICE)

Method	Syntax	Meaning
Append log	`echo "text" >> file.log`	Write to file (append)
Timestamp	`date '+%F %T'`	Current date & time
Tee	`cmd | tee file.log`	Print + write to file
Function	reusable logging	Structured & reusable

Example:

```
echo "$(date '+%F %T') Server started" >> log.txt
```

Reusable Functions:

```
log() {
  echo "$(date '+%F %T') - $1"
}

log "App started"
```

Better (structured):

```
log_info()  { echo "[INFO]  $(date '+%F %T') - $1"; }
log_error() { echo "[ERROR] $(date '+%F %T') - $1"; }

log_info "App started"
log_error "Something failed"
```

Advanced (safer than echo):

```
printf "%s - %s\n" "$(date '+%F %T')" "message"
```

6. FILE OPS / BACKUP / CLEANUP

Task	Command	Meaning
Backup	`tar -czf file.tar.gz dir/`	Compress archive
Copy backup	`cp -r dir backup/`	Simple copy
Sync	`rsync -av src/ dest/`	Efficient transfer
Delete old	`find . -type f -mtime +7 -delete`	Safe cleanup
Compress logs	`find . -name "*.log" -exec grip {} \;`	Save space

7. PIPELINE CORE (LOG PROCESSING)

```
grep → awk → sort → uniq -c → sort -nr → head
```

Example:

```
grep "ERROR" log.txt | awk '{print $5}' | sort | \
uniq -c | sort -nr | head
```

Meaning:

- Filter → extract → group → count → rank → top

Note:

- awk field ($5) depends on log format

8. USEFUL ONE-LINERS

Task	Command
Delete old files	`find . -type f -mtime +7 -delete`
Disk usage	`df -h`
Live logs	`tail -f log.txt`
Top 10 largest files	`du -ah | sort -hr | head -10`
Find big files (>100M)	`find . -type f -size +100M -exec ls -lh {} \;`

9. LOG ANALYZER (PROJECT PATTERN)

Task	Command
Count errors	`grep -i error log.txt`
Critical lines	`grep -n CRITICAL log.txt`
Top errors	`grep -i error log.txt | awk '{print $5}' | sort | \ uniq -c | sort -nr | head`

Flow:

Generate → Analyze → Report → Archive

10. LOG GENERATOR (TESTING)

```
for i in {1..10}; do
    echo "$RANDOM ERROR occurred" >> log.txt
done
```

11. ARCHIVE & REPORT

Task	Command	Meaning
Create dir	`mkdir -p archive`	Create directory if not exists
Move file	`mv file archive/`	Move file to archive
Timestamp	`$(date +%F)`	Date (YYYY-MM-DD)

Example:

```
mkdir -p archive
mv log.txt archive/log_$(date +%F).txt
```

12. BACKUP STRATEGY

```
tar -czf backup_$(date +%F).tar.gz dir/
```

Tips:

- Use compression to save space
- Include date in filename
- Store backups in separate location (if possible)

13. DEVOPS SYSTEM COMMANDS

Resource	Command	Meaning
Disk	`df -h`	Filesystem disk usage
Memory	`free -h`	RAM usage
CPU / Process	`ps aux`	List running processes
Service status	`systemctl is-active <service>`	Check if service is active
Service detailed	`systemctl status <service>`	Detailed service status

Optional (detailed status):

```
systemctl status <service>
```

FINAL MEMORY BLOCK (ULTIMATE)

SOURCE
- source → same shell
- ./script / bash → subshell

CHECKS
- $EUID → root
- command -v → dependency

IDEMPOTENT
- check || install
- skip if exists

LOGGING
- echo >> file
- tee → print + log
- use timestamp

FILES
- tar → backup
- find → cleanup (use -type f)
  rsync → sync

PIPELINE
- grep → awk
- sort → uniq -c
- rspt → head

OPS
- tail -f | grep → live logs

PROJECT FLOW

Generate → Analyze → Report → Archive

ERROR / SAFETY
- set -euo pipefail
- cmd || → fallback

TRAP
- trap EXIT → cleanup
- trap SIGINT → Ctrl+C

ADVANCED
- (cmd) → subshell
- <(cmd) → process substitution
- exec 3> → file descriptor

FINAL NOTES (IMPORTANT)

- bash script.sh → same as ./script.sh (subshell)
- awk fields depend on log format
- always use -type f with find before delete
- cron runs with minimal environment (no PATH, aliases, etc.)
- prefer absolute paths in scripts & cron jobs

Safe delete example:

```
find /var/log/myapp -type f -mtime +7 -delete
```

BASH ADVANCED CHEATSHEET

PART 1 — BASH CORE (FOUNDATION + SAFE SCRIPTING)

Focus: How Bash works + writing safe scripts

1 Globbing vs Regex (IMPORTANT)

```
*.log          # glob (shell expands)
grep ".*log"   # regex (grep interprets)
```

- Shell expands * before the command runs
- grep uses regex, not glob patterns

2 Brace Expansion

```
echo file{1..3}.txt
# file1.txt file2.txt file3.txt
```

- Happens before execution (handled by shell)
- Useful for loops, bulk file creation

3 Parameter Expansion (Powerful & Fast)

```
name="devops"
echo "${name^^}"          # uppercase
echo "${name,,}"          # lowercase
echo "${name:0:3}"        # substring
echo "${name/dev/linux}"  # replace first match
```

- Faster and cleaner than sed / awk for simple operations

4 Default & Required Variables (Safe Usage)

```
echo "${VAR:-default}"   # use default if unset OR empty
echo "${VAR-default}"    # use default only if UNSET
echo "${VAR:?missing}"   # exit with error if unset or empty
```

5 Arithmetic

```
((a = 5 + 3))   # ✓ preferred
let a=5+3       # ⚠ legacy style
```

- Prefer $(( )) or (( )) over let

6 Safe Input Handling (read)

```
read -r line     # prevents backslash escaping
read -t 5 var    # timeout (seconds)
read -s pass     # silent (password)
```

- Always use -r unless escape processing is needed

7 Safe Looping Over Files

❌ Unsafe:

```
for f in $(ls *.log)
```

✓ Correct:

```
for f in *.log
```

- ⚠ Without nullglob, if no match → loop runs once with literal "*.log"
- Avoid word-splitting and issues with spaces

8 Null-Safe Globbing

```
shopt -s nullglob
```

- Prevents unmatched patterns like *.log from being treated as literals
- Loop will skip if no files match
- ⚠ Affects entire script/session unless unset (shopt -u nullglob)

9 Quoting Arguments Properly

```
"$@"   # ✓ preserves arguments correctly
$@     # ⚠ causes word splitting
```

10 Command Control

```
command ls     # bypass alias/function (uses command from $PATH)
builtin echo   # force shell builtin
```

- Ensures predictable behavior in scripts

11 Exit vs Return

```
exit 1     # exits entire script
return 1   # exits function
(only valid inside functions)
```

12 Timing & Performance

```
time ls
```

- Shows execution time (real/user/sys)
- ⚠ Output format depends on shell

13 File Descriptors (Advanced Intro)

```
exec 3>file.txt    # 3 = custom FD
echo "hello" >&3
exec 3>&-
```

- Used for custom logging and advanced redirection

14 File Test Extensions

```
[ -L file ]            # is symlink
[ file1 -nt file2 ]    # newer than
[ file1 -ot file2 ]    # older than
[ file1 -ef file2 ]    # same inode
```

- Common in backup / deploy scripts

15 String Pattern Matching (Bash Only)

```
[[ $name == dev* ]]      # starts with
[[ $file == *.log ]]     # ends with
[[ $msg == *error* ]]    # contains
```

- Works in [[ ]] (Bash keyword)
- Safer: no word splitting, supports pattern matching
- [ ] is POSIX test (more limited)

16 CLI Arguments Parsing (getopts)

```
while getopts "f:n:" opt; do
  case $opt in
    f) file=$OPTARG ;;
    n) num=$OPTARG ;;
  esac
done
```

- : means the option expects value

17 Interactive Menu (select)

```
select opt in start stop exit; do
   echo "You chose $opt"
   [[ $opt == exit ]] && break
done
```

- Simple interactive menu (loops until explicitly broken)

19 Directory & Path Utilities

```
dirname  "$0"      # script directory (may be relative)
basename "$0"      # script name
realpath file      # absolute path (may not exist on all systems)
readlink -f file   # alternative (GNU/Linux only, not macOS by default)
```

- ⚠ dirname "$0" may not return absolute path

19 Environment Debugging

```
env        # all environment variables
printenv   # exported variables only
```

20 Safe Temporary Files

```
mktemp
```

- Avoids race conditions vs manual temp filenames

21 Empty Variable Handling

```
[ -z "$var" ]   # true if empty OR unset
[ -n "$var" ]   # true if not empty
```

- Always quote variables to avoid errors

QUICK SUMMARY — KEY TAKEAWAYS

- **Globbing vs Regex** — Shell vs command (glob ≠ regex)
- **Expansion** — Brace & parameter expansion are powerful
- **Safe Scripting** — Use -r, nullglob, quote "$@", avoid $(ls)
- **Control** — command, builtin, exit vs return, file descriptors
- **Parsing & Menus** — getopts for CLI args, select for interactive menus
- **Debug & Paths** — env/printenv, path tools for diagnostics
- **Safety First** — mktemp, tests, proper quoting, avoid pitfalls

ℹ Write safe, portable, and efficient Bash scripts!

Happy Scripting! 🚀

BASH ADVANCED CHEATSHEET

PART 2 — DEVOPS & SYSTEM (REAL-WORLD USAGE)

Focus: How Bash is used in automation, cron, CI/CD

1 Process Finding (Cleaner)

```
pgrep nginx      # matches any process containing "nginx"
pgrep -x nginx   # exact match
pgrep -f nginx   # match full command line
```

- Cleaner than `ps | grep` (avoids matching the grep process itself)

2 Background Jobs

```
cmd &            # run in background
wait             # wait for all background jobs
wait <pid>       # wait for specific job
```

- ⚠ `wait` without PID waits for all jobs

3 Parallel Execution

```
echo "a b c d" | xargs -n1 -P 4 echo
# demo (4 parallel processes)
```

```
find . -name "*.log" -print0 |
  xargs -0 -n1 -P 4 gzip
# practical example
```

- `xargs -P` controls parallelism

4 Locking (Prevent Duplicate Runs)

```
flock /tmp/script.lock -c "./script.sh"
```

- Ensures only one instance runs at a time
- Common in cron jobs
- ⚠ This spawns a subshell

Alternative (same process locking):

```
exec 200>/tmp/script.lock
flock -n 200 || exit 1
```

- ✔ No subshell
- ✔ -n → fail immediately if locked

5 Subshell Behavior

```
(cd /tmp)     # runs in subshell (isolated)
cd /tmp       # affects current shell/script
```

- Changes inside subshell do not persist

6 Process Substitution (Advanced)

```
diff <(ls dir1) <(ls dir2)
```

- Treats command output as file-like input
- Useful for inline comparisons
- ⚠ Bash/Zsh feature (not POSIX sh)
- ⚠ Requires /dev/fd support (most Linux systems)

7 Signal Handling (trap)

```
trap 'echo "Interrupted"; exit 1' SIGINT
```

- Handles signals like Ctrl+C gracefully

Cleanup pattern (recommended):

```
trap 'rm -f /tmp/file' EXIT
```

- ✔ Runs on script exit (success or failure)
- ✔ Useful for temp files, locks

8 xargs (Input → Arguments)

```
find . -name "*.log" | xargs rm
```

- ⚠ Unsafe with spaces/newlines

Safe version:

```
find . -name "*.log" -print0 | xargs -0 rm
```

Prevent empty execution (GNU):

```
find . -name "*.log" -print0 | xargs -0 -r rm
```

9 tee (Dual Output)

```
echo "hi" | tee file.txt      # overwrite
echo "hi" | tee -a file.txt   # append
```

- Writes to file AND stdout

10 Export (Environment Scope)

```
export VAR=value
```

- Makes variable available to child processes only
- Does NOT affect parent shell

11 Advanced Empty Check

```
[ -z "${var+x}" ]    # true if UNSET only
```

- Distinguishes between unset vs empty

QUICK REFERENCE SUMMARY — PART 2

Processes pgrep is cleaner than ps | grep	**Jobs** Run in background & wait properly	**Parallelism** xargs -P for parallel execution	**Locking** flock prevents duplicate runs	**Signals** trap for graceful handling & cleanup
xargs Use -print0 and -0 for safety	**tee** Write to file & screen	**Export** Child processes inherit only	**Subshells** (...) runs isolated from current shell	**Check Vars** Use ${var+x} to check if unset

★ Use these advanced techniques to write robust, safe, and production-ready Bash scripts!

Happy Scripting! 🚀

1. Permission Basics

Topic	Syntax	Example
Read	r = 4	view file
Write	w = 2	modify file
Execute	x = 1	run file

2. Numeric to Symbolic Mapping

Topic	Syntax	Example
Full permission	7 = 4+2+1	rwx
Read + Write	6 = 4+2	rw-
Read + Execute	5 = 4+1	r-x
Read only	4	r--
Write + Execute	3	-wx
Write only	2	-w-
Execute only	1	--x
No permission	0	---

3. User Categories

Topic	Syntax	Example
Owner	first digit	7 in 755
Group	second digit	5 in 755
Others	third digit	5 in 755

4. chmod (Numeric)

Topic	Syntax	Example
Set permissions	chmod XYZ file	chmod 755 script.sh
Full access	chmod 777 file	everyone rwx
Secure file	chmod 644 file	rw-r--r--

5. chmod (Symbolic)

Topic	Syntax	Example
Add permisbion	chmod +x file	make executable
Remove permission	chmod -w file	remove write
User specific	chmod u+x file	owner execute
Group specific	chmod g-w file	group remove write
Others	chmod o+r file	others read

6. Real Examples

Topic	Syntax	Example
Script executable	chmod 755 script.sh	run script
Private file	chmod 600 file.txt	only owner
Public read	chmod 644 file.txt	common default

7. Directory Permissions (Important nuance)

Topic	Syntax	Example
Read dir	r	list files
Write dir	w	create/delete
Execute dir	x	enter (cd)

🔑 **Key insight:** Without x → you cannot cd into directory even if r exists

8. Check Permissions

Topic	Syntax	Example
List details	ls -l	shows -rwxr-xr-x

9. Ownership

Topic	Syntax	Example
Change owner	chown user file	chown ubuntu file
Change group	chown :group file	chown :devops file
Both	chown user:group file	full change

10. FINAL UNDERSTANDING

Example:

```
chmod 755 script.sh
```

Breakdown:

Role	Value	Permission
Owner	7	rwx
Group	5	r-x
Others	5	r-x

11. COMMON MISTAKE

```
chmod 777 file
```

⚠️ Overly permissive! Everyone can read, write, and execute.

💡 r = 4, w = 2, x = 1
→ add to get number